

7 ファイルシステム

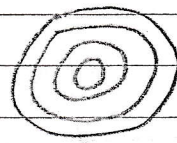
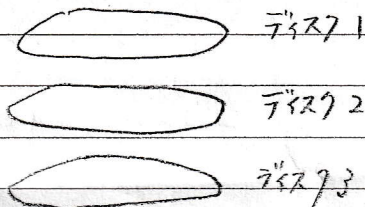
主記憶 揮発性

二次記憶 不揮発性

↳ ディスク, CD-ROM等

7.1 ハードディスクの構造

ハードディスクの内部構造



← 同心円状に区分されている

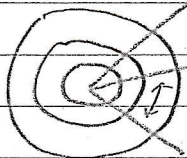
1つのドーナツ状の部分

をトラックと呼ぶ

各ディスクの同じトラック番号のトラックを

シリンダと呼ぶ

各トラックはディスクの中心から放射線状に等分割されている



等分割された部分をセクタと呼ぶ

セクタはハードディスクがデータを読み書きするための最小単位

セクタを読み書きするための磁気ヘッドがあり、ヘッドを読み書きしたいセクタへ移動するために、シーク (seek) と言う

OSは連続したいくつかのセクタをまとめて固定長のブロックという単位でディスクにアクセスする比较多い

セクタのサイズ (512バイト ~ 2Kバイト)

シーブに比べ時間からセグを読み書きする時間を比べると長い

おおよそ 100 倍

トラックを飛ばして移動、ヘッドを回転してセグを移動

外部部ほどセグ数を増やすことも行われている。

7.2 ファイルシステムの仕組み

ファイル : データのかたち + データに付随する情報
(生成日時, 所有者, サイズ, アクセス権限)

ファイルシステム : (厳密には) 二次記憶装置を抽象化したもので、
二次記憶装置上にデータを保管し、保管されたデータへのアクセス手段を提供する仕組み

要は、ファイルにどのようなデータに関する情報を付随させるか、
セグへの書き込み、読み出しをどのように行うのか、
ファイルをどのように管理するのか。

↓
どうやって決められているか

ファイルシステムを利用するメリット

- ・ファイル名によるアクセス → セグをいちいち扱う必要がなくなる。
- ・ファイルの作成、伸長

OSがセグの空き状況を管理しているので、容易にできる

- ・ファイルの保護

どのファイルにアクセスできるか、管理できるか
誰か

7.2.2 ファイルに対する操作

ファイルに対する操作は全システムコールにより行われる

OPEN

ファイル名を引数とする

OS内部でファイル名で指定されたファイルの二次記憶装置上での位置を特定し、アクセス権の有無をチェックする。

オープンされたファイルにアクセスするのに必要なカーネル内のデータ領域の確保や初期化などを行う。

CLOSE

ファイルを閉じる。ファイルのオープン時に確保した領域の解放などの後処理を行う。

READ

ファイルからメモリにデータを読み込む。通常、前に読み込んで終わった位置から続きを読む。

WRITE

メモリからファイルにデータを書き込む。通常、前に書き込んで終わった位置から続きを書く。

SEEK

ファイルを読み書きする位置を変更する

7.2.4 ファイルの管理情報

ファイルシステムによってファイルの管理情報は異なる。一般的に以下の様な情報を含む。

ファイル名

ファイルの属性 テキストファイル

ファイルの存在位置 どこセクタにあるのか

ファイルの大きさ

ファイルの保護情報 Unix での r, w, x

ファイルのアクセス時間 最終アクセス時刻, 更新時刻, 作成時刻

5 相互排除と同期

5.2 競合状態

複数のスレッドを用いるマルチスレッドプログラムでは、スレッド間で共有する変数・データにアクセスする場合には、競合状態が生じ得る

race condition
→ プログラムが意図していない

状態

実行の順序

結果が異なる

Load a
 $a \leftarrow a + 1$

X-メモリからデータを読み込み

STORE a

X-メモリにデータを書き込み

このプログラムを2つのスレッドで同時に実行 (aの初期値は0)

別々のスレッドに属する命令列が入り交じりながら実行される

→ インターリーブ (interleave)

別のスレッドの STORE 操作が完了する前に、Load の操作が行われる
場合がある



タイミングで結果がかわる

2つのプロセスが同じファイルに同時にアクセスする場合に競合状態になる
可能性がある

5.3 相互排除の問題

競合状態が生じる要因は、複数のプロセスやスレッドが共有データ (リソース) に同時にアクセス (2つ以上) するため

共有データにアクセスするプロセスやスレッドが常に (1つだけ) に制御される
べき

プログラム中において 共有データにアクセスする部分を

クリティカル セクション (critical section)

をよぶ

クリティカルセクションを実行するスレッドをひとつに限定する問題

↳ 相互排除 (mutual exclusion, mutex)
 " 排他制御 の問題")

5.4 排他制御のためのハードウェア支援

競合状態が起る要因のひとつは、メモリから値を読み出し、ふたたび
 メモリに値を書き込む間に「インターリーブ」が起きたため

値の読み出しから書き込みをひとつの機械語命令として実行すれば、
 インターリーブを防ぐことができる

この
 説明 test and set 命令

$$\begin{matrix} L_i = C \\ C = 1 \end{matrix} \quad \left\{ \begin{array}{l} \text{この処理をインターリーブせずに atomic 実行} \end{array} \right.$$

C : スレッド間の共有変数

L_i : i 番目のスレッドのローカル変数 (当該のスレッドしかアクセスできない)

一連の処理がインターリーブされずに実行されることを atomic
 に実行されるという

部屋の扉を確認 → 入室

他の人が確認し、入室済みならば

5.4.2 exchange 命令

変数 A と変数 B の値を入れ換える処理を不可分に実行する

1. temp = A
2. A = B
3. B = temp

但し、前の使用中、使用可のその内容を見るという動作と
内容を変えるという動作を
アトミックにできる

↳ 排他制御できる

5.5 ロック

最近の OS ではプロセス間やスレッド間の排他制御を行うために、
ロック (lock) という扱いやすき機能を提供している

部屋に鍵をかけたイメージ

Lock という型の変数 (錠前のようなもの)

Lock 変数に対しては、以下の 2 つの関数を通じて操作のみが可能である

- | | |
|---------------------------|-------|
| 1 void Lock (Lock lock) | 鍵をかける |
| 2 void Unlock (Lock lock) | 鍵をはずす |

Lock (lock) を実行すると、Lock 型の変数 lock に鍵をかける。
もし、すでに鍵がかけられている場合には、鍵が閉(まって待たされる)。

↳ suspend するといふ

クリティカルセクションを抜けるために Unlock (lock) により鍵をはずす

ロックの集約

test and set 命令を用いて、ロックがとられているか check

すでにロックがとられている場合、スレッドを休止させ、ロック解放までの行列に加入する

Unlock() では、解放待ちのスレッドがあるか調べ、そのうち1つを起しにロックを付与。ロック待ちのスレッドがある場合は、そのスレッドもロックを確保していることを記録する

5.6 セマフォ

セマフォ (semaphore)

ロックの機能を強化

ロック: 1人しか入れない部屋

セマフォ: 5人が入れない部屋

セマフォを使う場合、初期値が必要 (5: 部屋の利用人数)

セマフォに対して許されている操作

1. void Wait (Semaphore S);
2. void Signal (Semaphore S);

Wait: Sを1減らし、部屋に入る。部屋が満員の場合待つ

Signal: 部屋を待っている人が入れば1人、自分自身退去待ちになっている場合は、Sの数を1増やす